

Stack Safety

By Jonathan Louie

Agenda

What is Stack Safety?

My problem

Possible solutions

How I fixed it

What is Stack Safety?

Stack Safety

- **Safety against stack overflows**
- **Recursive functions commonly cause stack overflows**

My problem

My problem

- **Multiple recursive functions (substitution, type checking, codegen)**
- **Stack overflows compiling 400 lines of code**

Possible solutions

Tail-call Optimisation

- If a recursive call is the last operation before a function returns, the compiler can reuse the stack frame
- Problem: Not guaranteed by Rust at all

Rewrite as an iterative loop

- I tried this: https://github.com/jonathanrlouie/delver-script/tree/stack_overflows

What went wrong?

- **The logic became quite convoluted and more prone to errors**
- **The stack overflows were still happening!**

How?

- <https://fast.github.io/blog/stacksafe-taming-recursion-in-rust-without-stack-overflow/>
 - “Derived traits like `Debug`, `Clone`, and `Drop` on deeply nested structures still cause stack overflow, you must manually implement all traits with stack protection”

How I fixed it

stacksafe

- <https://github.com/fast/stacksafe>
- Easiest solution for me, fast to implement

Trade-offs?

- For functions that can be easily converted to iterative loops, you will allocate more memory to grow the stack
 - I was already allocating memory though in the iterative version
- `#[stacksafe]` Annotations and `.into()` calls everywhere
 - A little tedious to implement, but still saved me a lot of time and headache

Some potential alternatives

- <https://crates.io/crates/stacker>
 - `stacksafe` is built on top of this one
 - It just allows you to expand stack size by allocating
- <https://crates.io/crates/recursive>
- Neither of these worked for me because of the `Clone` issue

Key takeaways

- **There's no tail-call optimisation in Rust, so prefer loops**
- **Default derive trait impls are recursive and can stack overflow**
- **stacksafe crate provides a pretty quick and easy way to make your recursive programs stack safe**